

Custom Scripting Manual

Complete Guide for C# (Roslyn) & JavaScript (Jint)

FileFlow Studio — Motor Visual de Automatización DAG en .NET 9

FileFlow Studio — Custom Scripting Manual (C# & JavaScript)

Welcome to the official manual for the **Custom Script Node** (`CustomScriptNode`). This guide is designed step-by-step to empower any user, from beginner to advanced programmer, to write tailored scripts, build intelligent multi-port dispatchers, query pipeline metadata, and transform files without needing external IDEs.

Table of Contents

1. What is the Custom Script Node?
2. Choosing Between C# and JavaScript
3. The Script Studio Window
4. Creating and Managing Dynamic Ports (Inputs and Outputs)
5. Accessing File Information (`Item` / `file`)
6. Accessing Implicit Variables and Flow Metadata
7. How to Emit Files to Outputs (`EmitAsync` and `emit`)
8. Logging and Telemetry Messages (`Log` and `console.log`)
9. Catalog of 7 Ready-to-Use Practical Examples
10. Frequently Asked Questions and Troubleshooting

1. What is the Custom Script Node?

The **Custom Script** node is a programmable workstation positioned at any point along your FileFlow Studio visual pipeline. When a file arrives at the node, the engine executes your custom code, allowing you to:

- **Inspect and classify** files based on size, name, extension, or content.
- **Access rich metadata** generated by earlier nodes (calculated hashes, EXIF camera tags, PDF page counts, video duration, etc.).
- **Mutate properties and add Tags** that downstream nodes can inspect later.
- **Branch the workflow** by dispatching items to one or multiple custom output ports (e.g., `Approved` , `LargeFiles` , `Errors` , `Docs`).

2. Choosing Between C# and JavaScript

FileFlow Studio features **two independent execution runtimes**:

Feature	⚡ C# (Roslyn Scripting)	🌐 JavaScript (Jint Sandbox)
---	---	---
Target Audience	Maximum performance or familiarity with C# and .NET.	Developers preferring lightweight, flexible web-style syntax.
Performance	Ultra-fast (.NET 9 in-memory JIT compilation with SHA256 caching).	Fast and safe (Managed in-memory sandboxed engine).
Async Dispatch	<code>await EmitAsync("PortName");</code>	<code>emit("PortName", item);</code>
Best Suited For	Mathematical calculations, file system manipulations, and .NET types.	String manipulation, regular expressions, and JSON parsing.

Note: You can switch between languages at any time using the dropdown selector at the top of the Script Studio editor.

3. The Script Studio Window

To open the programming studio, click the  **Script Editor ...** button located on the node card in the visual canvas.

The window consists of 4 main areas:

1. **Top Bar:** Language selector (C# / JavaScript),  **Manual PDF ...** button, and **Built-in Templates** menu.
2. **Central Code Editor:** Syntax-highlighted editor with line numbers, code folding, and automatic theme adaptation.
3. **Right Side Panel:**
 -  **Ports Tab:** Add or remove dynamic input and output ports with a single click.
 -  **Live Test Tab:** Simulate script execution against a synthetic file item and inspect emitted ports and log traces in real time.
 -  **Save Tab:** Save your favorite scripts into your personal library for reuse across workflows.
4. **Bottom Bar:** Green  **Apply and Close** button to synchronize changes with the visual graph.

4. Creating and Managing Dynamic Ports (Inputs and Outputs)

Unlike static nodes with hardcoded ports, in the script node **you define the ports**.

How to add a new port:

1. In the  **Ports** tab, type the desired name in the text box (e.g., `Compressed`).
2. Click the  **Add** button.
3. The port immediately appears in the list. When you click Apply, the connector is created on the node card in the canvas so you can connect a wire to the next node.

Naming guidelines for ports:

- Use clean, descriptive names without special symbols (e.g., `Out`, `True`, `False`, `LargeFiles`, `Videos`, `PDFs`, `Quarantine`).
- The standard default output port is named `Out`.

5. Accessing File Information (`Item` / `file`)

Inside your script, the incoming file context is accessible via the global variable `Item` (or `file`).

Core Available Properties

Property	Type	Description	C# Usage Example	JavaScript Usage Example
---	---	---	---	---
<code>Item.FileName</code>	<code>string</code>	File name including extension.	<code>string n = Item.FileName;</code>	<code>let n = item.FileName;</code>
<code>Item.CurrentPath</code>	<code>string</code>	Current on-disk path at this step.	<code>string p = Item.CurrentPath;</code>	<code>let p = item.CurrentPath;</code>
<code>Item.OriginalPath</code>	<code>string</code>	Immutable original entry path.	<code>string o = Item.OriginalPath;</code>	<code>let o = item.OriginalPath;</code>
<code>Item.FileSizeBytes</code>	<code>long</code>	Exact file size in bytes.	<code>long bytes = Item.FileSizeBytes;</code>	<code>let bytes = item.FileSizeBytes;</code>
<code>Item.IsDirectory</code>	<code>bool</code>	Whether the item is a folder.	<code>if (Item.IsDirectory) ...</code>	<code>if (item.IsDirectory) ...</code>
<code>Item.Metadata</code>	<code>Dictionary</code>	Key-value metadata dictionary.	<code>Item.Metadata["Key"] = 123;</code>	<code>item.Metadata['Key'] = 123;</code>
<code>Item.Tags</code>	<code>HashSet</code>	Collection of string labels/tags.	<code>Item.Tags.Add("Verified");</code>	<code>item.Tags.Add("Verified");</code>

6. Accessing Implicit Variables and Flow Metadata

FileFlow Studio automatically propagates metadata across DAG nodes. You can access this data in three ways:

1. Direct Access to `Item.Metadata` Dictionary

If earlier nodes computed a SHA-256 hash or extracted EXIF camera attributes, you can read them directly:

```
// In C#:
if (Item.Metadata.TryGetValue("Hash:SHA256", out var hash))
{
    Log($"SHA256 checksum is: {hash}");
}
```

```
// In JavaScript:
if (item.Metadata['Hash:SHA256']) {
    log("SHA256 checksum is: " + item.Metadata['Hash:SHA256']);
}
```

2. Universal Template Function `Resolve("{Template}")`

Resolve any FileFlow token or expression using `Resolve( ... )` (in C#) or `resolve( ... )` (in JS):

```
// In C#:
string formattedName = Resolve("{Year}-{Month}_{FileNameNoExt}_{OK}.{Ext}");
string sizeMessage = Resolve("File size is {SizeMB} MB");
```

```
// In JavaScript:
let formattedName = resolve("{Year}-{Month}_{FileNameNoExt}_{OK}.{Ext}");
let currentUser = resolve("{UserName}");
```

3. Implicit Token Catalog Reference

Token	Description	Example Output
<code>---</code>	<code>---</code>	<code>---</code>
<code>{FileName}</code>	File name with extension.	<code>holiday_photo.jpg</code>
<code>{FileNameNoExt}</code>	File name without extension.	<code>holiday_photo</code>
<code>{Ext}</code> or <code>{Extension}</code>	Extension without dot.	<code>jpg</code>
<code>{SizeMB}</code>	Formatted size in Megabytes.	<code>14.50</code>
<code>{SizeBytes}</code>	Exact integer byte count.	<code>15204352</code>
<code>{SizeGB}</code>	Formatted size in Gigabytes.	<code>1.42</code>
<code>{Year}</code> , <code>{Month}</code> , <code>{Day}</code>	Current year, month, and day.	<code>2026</code> , <code>09</code> , <code>02</code>
<code>{Date:yyyy-MM-dd}</code>	Formatted date string.	<code>2026-09-02</code>
<code>{UserName}</code>	Current Windows username.	<code>kaoticos53</code>
<code>{MachineName}</code>	Machine host name.	<code>DESKTOP-PRO</code>
<code>{Hash:SHA256}</code>	SHA-256 hash (if calculated).	<code>e3b0c44298fc1c ...</code>
<code>{Exif:CameraModel}</code>	EXIF camera model (if image).	<code>Sony ILCE-7M4</code>
<code>{Doc:PageCount}</code>	Page count (if PDF/Doc).	<code>18</code>
<code>{Media:Duration}</code>	Audio or video duration.	<code>00:45:12</code>

7. How to Emit Files to Outputs (`EmitAsync` and `emit`)

For the file to continue downstream to connected nodes, **you must emit it to one or more output ports.**

C# Syntax:

```
// Emit to default output port "Out"
await EmitAsync("Out");

// Emit to a custom port
await EmitAsync("Approved");

// Multi-port parallel branch dispatch
await EmitAsync("LocalCopy");
await EmitAsync("CloudBackup");
```

JavaScript Syntax:

```
// Emit to default output port "Out"
emit("Out", item);

// Emit to a custom port
emit("Approved", item);

// Conditional routing
if (item.FileSizeBytes > 10485760) {
    emit("LargeFiles", item);
} else {
    emit("SmallFiles", item);
}
```

8. Logging and Telemetry Messages (`Log` and `console.log`)

Messages sent from your script appear in FileFlow Studio's bottom telemetry console and in the live tester:

- **In C#:**
 - `Log("Processing completed");` (*Information level*)
 - `Log("Warning: file size exceeds threshold", LogLevel.Warning);`
 - `Log("Validation error encountered", LogLevel.Error);`
- **In JavaScript:**
 - `console.log("Informational log");`
 - `console.warn("Warning trace");`
 - `console.error("Error trace");`
 - `log("Quick message");`

9. Catalog of 7 Ready-to-Use Practical Examples

🚩 Example 1 (C# - Basic): Megabyte Size Classifier

Required Output Ports: `LargeFiles` , `SmallFiles`

```
// Calculate size in Megabytes
double sizeMb = Item.FileSizeBytes / (1024.0 * 1024.0);

// Record calculated size in metadata for downstream nodes
Item.Metadata["Calculated_SizeMB"] = sizeMb;

if (sizeMb >= 100.0)
{
    Item.Tags.Add("Heavy");
    Log($"Large file detected ({sizeMb:F2} MB) -> Routing to LargeFiles");
    await EmitAsync("LargeFiles");
}
else
{
    Item.Tags.Add("Light");
    Log($"Small file detected ({sizeMb:F2} MB) -> Routing to SmallFiles");
    await EmitAsync("SmallFiles");
}
}
```

🚀 Example 2 (C# - Basic): Audit Metadata Injection

Required Output Ports: Out

```
// Inject system execution audit trail
Item.Metadata["Audit_User"] = Environment.UserName;
Item.Metadata["Audit_Machine"] = Environment.MachineName;
Item.Metadata["Audit_TimestampUtc"] = DateTime.UtcNow.ToString("yyyy-MM-dd HH:mm:ss");

Item.Tags.Add("Audited");
Log($"Audit metadata injected into {Item.FileName}");

await EmitAsync("Out");
```

🚀 Example 3 (C# - Intermediate): Multi-Extension Content Router

Required Output Ports: Images , Videos , Documents , Other

```

var ext = Path.GetExtension(Item.FileName).ToLowerInvariant();

var images = new[] { ".jpg", ".jpeg", ".png", ".webp", ".gif", ".raw" };
var videos = new[] { ".mp4", ".mkv", ".mov", ".avi", ".webm" };
var docs = new[] { ".pdf", ".docx", ".xlsx", ".txt", ".epub" };

if (images.Contains(ext))
{
    Item.Metadata["ContentType"] = "Image";
    await EmitAsync("Images");
}
else if (videos.Contains(ext))
{
    Item.Metadata["ContentType"] = "Video";
    await EmitAsync("Videos");
}
else if (docs.Contains(ext))
{
    Item.Metadata["ContentType"] = "Document";
    await EmitAsync("Documents");
}
else
{
    Item.Metadata["ContentType"] = "Other";
    await EmitAsync("Other");
}

```

✦ Example 4 (JavaScript - Basic): Name Sanitizer & Validator

Required Output Ports: `Valid`, `NeedsSanitization`

```

var name = item.FileName;

// Check for double spaces or illegal symbols
var hasDoubleSpaces = name.indexOf(' ') !== -1;
var hasIllegalChars = /[!@?;|!#$%&*]/.test(name);

if (hasDoubleSpaces || hasIllegalChars) {
    console.warn("File name contains invalid characters: " + name);
    item.Metadata["SuggestedName"] = name.replace(/\\s+/g, '_').replace(/[!@?;|!#$%&*]/g, '');
    item.Tags.Add("SanitizeRequired");
    emit("NeedsSanitization", item);
} else {
    emit("Valid", item);
}

```

✦ Example 5 (JavaScript - Intermediate): Date Template Media Routing

Required Output Ports: `MediaFiles`, `Other`


```

var ext = item.FileName.split('.').pop().toLowerCase();
var isMedia = ['mp4', 'mkv', 'mp3', 'wav', 'flac'].indexOf(ext) !== -1;

if (isMedia) {
    // Resolve date folder path dynamically
    var targetFolder = resolve("{Year}/{Month}/Media");
    item.Metadata["OrganizedFolder"] = targetFolder;

    console.log("Media file organized under: " + targetFolder);
    emit("MediaFiles", item);
} else {
    emit("Other", item);
}

```

✦ Example 6 (C# - Advanced): Upstream SHA-256 Integrity Verification

Required Output Ports: Verified, MissingChecksum

```

// Verify if an upstream node (HashCalculatorNode) computed SHA256
if (Item.Metadata.TryGetValue("Hash:SHA256", out var hashObj) && hashObj != null)
{
    string hash = hashObj.ToString();
    Log($"Verified checksum: {hash[..8]}... for {Item.FileName}");

    Item.Metadata["IntegrityStatus"] = "Passed";
    Item.Tags.Add("ChecksumVerified");

    await EmitAsync("Verified");
}
else
{
    Log($"Warning: {Item.FileName} has no upstream hash", LogLevel.Warning);
    Item.Metadata["IntegrityStatus"] = "Missing";
    await EmitAsync("MissingChecksum");
}

```

✦ Example 7 (JavaScript - Advanced): Document Classification by Page Count

Required Output Ports: ShortReports, LongBooks, NonDocument

```

// Check if DocumentProcessorNode extracted page count
if (item.Metadata['Doc:PageCount']) {
    var pages = parseInt(item.Metadata['Doc:PageCount'], 10);

    if (pages <= 10) {
        item.Tags.Add("ShortReport");
        emit("ShortReports", item);
    } else {
        item.Tags.Add("LongBook");
        emit("LongBooks", item);
    }
} else {
    emit("NonDocument", item);
}

```

10. Frequently Asked Questions and Troubleshooting

? What happens if my script does not call `EmitAsync` or `emit` ?

If the script does not emit the file to any port, the item is treated as filtered/dropped and will not travel to subsequent nodes.

? What does the error `CS1002: ; expected` mean?

In C#, every statement must end with a semicolon `;`. Ensure all lines conclude with `;`.

? What happens if a script runs too long or hangs?

For reliability and safety, the script node enforces a **Timeout (default 30 seconds)**. If a script encounters an infinite loop, execution is terminated and a clean error is recorded without freezing the UI.

? Can I emit an item to multiple outputs simultaneously?

Yes. You can invoke `await EmitAsync("Out1");` and `await EmitAsync("Out2");` in C# or `emit("Out1", item);` and `emit("Out2", item);` in JavaScript to duplicate the flow into parallel branches.

FileFlow Studio — Official Custom Scripting Documentation.